

Практикум по асинхронности в Python

20 заданий от базовых корутин до FastAPI-сервисов с очередями, streaming и отменой задач

Аудитория	Студенты со средним знанием Python: функции, классы, исключения, файлы и основы HTTP.
Стек	Стандартная библиотека Python для консольных приложений; FastAPI и Uvicorn для веб-приложений.
Результат	20 самостоятельных работ, разделённых на 4 уровня сложности по 5 заданий.

Версия для учебного курса ООП и асинхронного программирования

Оглавление

- Цели практикума
 - Требования к окружению
 - Правила сдачи
 - Уровень 1. Базовые конструкции: задания 1-5
 - Уровень 2. Управление конкурентностью: задания 6-10
 - Уровень 3. Координация и устойчивость: задания 11-15
 - Уровень 4. Комплексные мини-проекты: задания 16-20
 - Критерии итоговой оценки
 - Приложение. Команды запуска и проверки
-

Цели практикума

- Сформировать практическое понимание того, где асинхронность ускоряет программу, а где не помогает.
- Научить писать консольные `asyncio`-приложения с корректным жизненным циклом задач.
- Показать базовые и продвинутые способы координации: `Queue`, `Semaphore`, `Lock`, `timeout`, отмена и `streaming`.
- Перенести эти идеи в простые FastAPI-приложения без внешних брокеров, баз данных и дополнительных асинхронных библиотек.
- Закрепить инженерную дисциплину: измерять время, валидировать входные данные, завершать фоновые задачи и проверять отказные сценарии.

Требования к окружению

- Python 3.11 или новее. Для задания 17 требуется `asyncio.TaskGroup`; при более старой версии Python задание нужно заменить на вариант с `create_task` и `gather`.
- Для консольных заданий используется стандартная библиотека Python: `asyncio`, `time`, `argparse`, `uuid`, `urllib.request`, `json`, `signal` и близкие модули.
- Для веб-заданий разрешены FastAPI и Uvicorn. Обязательная часть не использует `httpx`, `aiofiles`, Redis, базы данных, Celery или внешние очереди.
- Рекомендуемые команды установки веб-части: `python -m venv .venv`, `source .venv/bin/activate`, `python -m pip install fastapi uvicorn`.
- Все программы должны запускаться из терминала и иметь понятные сообщения об ошибках для некорректного пользовательского ввода.

Правила сдачи

- Каждое задание сдаётся отдельным файлом или небольшой папкой с понятным именем.
- К каждому заданию приложите краткий `README.md`: что реализовано, как запустить, какие проверки выполнялись.

- Если в задании есть веб-сервис, приложите примеры HTTP-запросов через Swagger UI, `curl` или другой простой клиент.
- Не подменяйте асинхронность потоками там, где требуется `asyncio`; `asyncio.to\_thread` разрешён только для явно блокирующих операций стандартной библиотеки.
- Код должен завершаться без предупреждений о незакрытых задачах, соединениях и асинхронных генераторах.

Уровень 1. Базовые конструкции

Студент осваивает корутины, точки ожидания, простое конкурентное выполнение, очереди и первые асинхронные endpoint-ы FastAPI.

Задание 1. Консольный асинхронный таймер задач

Формат: Консоль	Уровень: Уровень 1	Ключевые темы: <code>async def</code> , <code>await</code> , <code>asyncio.sleep</code> , <code>asyncio.run</code> , измерение времени.
-----------------	--------------------	---

Цель

Понять, что корутина не выполняется сама по себе, а выполнение нескольких ожидающих операций можно организовать конкурентно.

Что реализовать

- Создайте консольную программу `async_timer.py`.
- Опишите корутину `run_timer(name: str, delay: float)`, которая печатает старт, ждёт `delay` секунд и печатает завершение.
- Добавьте два режима: последовательный запуск трёх таймеров и конкурентный запуск тех же таймеров.
- Измерьте длительность каждого режима через `time.perf_counter()`.

Обязательные требования

- Точка входа программы должна быть синхронной: `if __name__ == "__main__": asyncio.run(main())`.
- Внутри корутин нельзя использовать `time.sleep`.
- Названия задач и задержки должны задаваться в одном списке данных, а не быть размазаны по коду.

Ожидаемое поведение

- Последовательный режим выполняется примерно за сумму задержек.
- Конкурентный режим выполняется примерно за максимальную задержку.
- В выводе видно, что конкурентные таймеры стартуют до завершения друг друга.

Проверка

- Запустить `python async_timer.py`.
- Проверить, что оба режима печатают одинаковые задачи.
- Сравнить время: конкурентный режим должен быть заметно быстрее последовательного.

Критерии оценки

- Корректное использование `asyncio.run` и `await`.
- Отсутствие блокирующего `time.sleep` в асинхронной части.
- Понятный вывод, по которому можно объяснить разницу между последовательным и конкурентным ожиданием.

Усложнение

Добавьте аргумент командной строки `--mode sequential|concurrent|both` и проверку некорректного режима.

Задание 2. Консольный симулятор загрузок

Формат: Консоль	Уровень: Уровень 1	Ключевые темы: <code>asyncio.create_task</code> , <code>asyncio.gather</code> , сбор результатов, порядок завершения.
-----------------	--------------------	---

Цель

Научиться создавать задачи явно и собирать результаты независимых операций.

Что реализовать

- Создайте `download_simulator.py`.
- Опишите корутину `download(url: str, delay: float) -> dict`, которая имитирует загрузку через `asyncio.sleep` и возвращает адрес, задержку и размер результата.
- Создайте 8-10 фиктивных URL с разными задержками.
- Запустите загрузки конкурентно через `create_task` и соберите результаты через `gather`.
- После завершения напечатайте таблицу результатов и общее время.

Обязательные требования

- Каждая задача должна иметь имя через `task.set_name()` или параметр `name` в `create_task`.
- Размер результата должен вычисляться детерминированно, например на основе длины URL.
- Нельзя обращаться к реальной сети: это именно симулятор.

Ожидаемое поведение

- Все загрузки стартуют почти сразу.
- Итоговый список результатов соответствует исходному набору URL.
- Общее время близко к самой большой задержке, а не к сумме задержек.

Проверка

- Запустить программу несколько раз и убедиться, что результат стабилен.
- Временно заменить конкурентный запуск на последовательный и сравнить время в отчёте.
- Проверить, что исключений `coroutine was never awaited` нет.

Критерии оценки

- Явное создание задач.
- Правильная работа с возвращаемыми значениями `gather`.
- Чистое разделение данных, корутины загрузки и функции вывода.

Усложнение

Добавьте режим, в котором одна загрузка завершается ошибкой, и покажите разницу между `gather(...)` и `gather(..., return_exceptions=True)`.

Задание 3. Консольная очередь сообщений

Формат: Консоль	Уровень: Уровень 1	Ключевые темы: <code>asyncio.Queue</code> , <code>producer/consumer</code> , <code>sentinel</code> , <code>task_done</code> , <code>join</code> .
-----------------	--------------------	---

Цель

Освоить передачу данных между корутинами без общего изменяемого списка.

Что реализовать

- Создайте `message_queue.py`.
- Реализуйте одного `producer`-а, который помещает в очередь 15 текстовых сообщений.
- Реализуйте двух `consumer`-ов, которые читают сообщения, имитируют обработку и печатают имя `worker`-а.
- После всех сообщений отправьте каждому `consumer`-у отдельный `sentinel` для завершения.
- Дождитесь обработки всех сообщений через `queue.join()`.

Обязательные требования

- Очередь должна иметь `maxsize`, например 5, чтобы была видна `backpressure`-семантика.
- `Consumer` обязан вызывать `queue.task_done()` для каждого обычного сообщения.
- `Sentinel` не должен обрабатываться как обычное сообщение.

Ожидаемое поведение

- Сообщения распределяются между двумя `consumer`-ами.
- Программа завершается сама, без бесконечного ожидания.
- При уменьшении `maxsize` `producer` иногда ждёт освобождения места.

Проверка

- Запустить `python message_queue.py`.
- Проверить, что обработаны все 15 сообщений.
- Добавить счётчик обработанных сообщений и вывести итоговую проверку.

Критерии оценки

- Корректный жизненный цикл `producer/consumer`.
- Нет зависания на `join`.
- Понятное использование `sentinel` и `task_done`.

Усложнение

Добавьте третий `consumer` и сравните распределение сообщений между `worker`-ами.

Задание 4. FastAPI-сервис задержек

Формат: FastAPI

Уровень: Уровень 1

Ключевые темы: `async endpoint`, `asyncio.sleep`, `Uvicorn`, `неблокирующее ожидание`.

Цель

Увидеть, как асинхронное ожидание внутри web endpoint-а позволяет серверу обслуживать несколько запросов одновременно.

Что реализовать

- Создайте `delay_api.py` с приложением `app = FastAPI()`.
- Добавьте `GET /ping`, который сразу возвращает `{"status": "ok"}`.
- Добавьте `GET /delay/{seconds}`, который проверяет диапазон `0 <= seconds <= 10`, ждёт через `asyncio.sleep(seconds)` и возвращает фактическое время ожидания.
- Добавьте краткую инструкцию запуска в комментарии в начале файла.

Обязательные требования

- Endpoint `/delay/{seconds}` должен быть объявлен через `async def`.
- Нельзя использовать `time.sleep`.
- Некорректное значение задержки должно возвращать HTTP 400.

Ожидаемое поведение

- `/ping` отвечает сразу.
- Параллельные запросы к `/delay/3` не блокируют быстрые запросы к `/ping`.
- Ответ содержит исходную задержку и измеренную длительность.

Проверка

- Запустить `uvicorn delay_api:app --reload`.
- Открыть `/docs` и выполнить `/ping` и `/delay/2`.
- В двух терминалах одновременно выполнить запросы к `/delay/3` и сравнить время.

Критерии оценки

- Правильная валидация входного параметра.
- Неплокирующее ожидание.
- Чёткие JSON-ответы без лишнего состояния.

Усложнение

Добавьте `GET /delay?seconds=...&label=...` и возвращайте `label` в ответе, сохранив `path`-вариант.

Задание 5. FastAPI мини-счётчик задач

Формат: FastAPI

Уровень: Уровень 1

Ключевые темы: `In-memory state`, `async endpoint`, `простая модель статусов`.

Цель

Построить первый web-сервис, который создаёт задачу, хранит её состояние в памяти и отдаёт статус по идентификатору.

Что реализовать

- Создайте `task_counter_api.py`.
- Добавьте `POST /tasks`, который принимает JSON с полем `title` и создаёт запись со статусом `created`.
- Добавьте `GET /tasks/{task_id}`, который возвращает задачу по идентификатору.
- Добавьте `GET /tasks`, который возвращает все созданные задачи.
- Храните задачи в словаре в памяти процесса.

Обязательные требования

- Идентификатор задачи должен быть строкой, например результатом `uuid.uuid4()`.
- Пустой `title` должен отклоняться.
- Для неизвестного `task_id` возвращайте HTTP 404.
- Не подключайте базу данных: цель задания - жизненный цикл состояния в памяти.

Ожидаемое поведение

- После `POST /tasks` задача доступна через `GET /tasks/{id}`.
- Список задач пополняется без перезапуска процесса.
- Ошибки валидации возвращаются понятным JSON-ответом.

Проверка

- Запустить `uvicorn task_counter_api:app --reload`.
- Создать две задачи через Swagger UI.
- Проверить получение каждой задачи и общий список.

Критерии оценки

- Корректные HTTP-коды.
- Простая и понятная структура хранения.
- Отсутствие глобальных side effect-ов кроме in-memory словаря.

Усложнение

Добавьте endpoint `PATCH /tasks/{task_id}/done`, который переводит задачу в статус `done`.

Уровень 2. Управление конкурентностью

Фокус на ограничении одновременности, отмене, тайм-аутах, TCP streams и переносе блокирующей работы за пределы event loop.

Задание 6. Консольный runner заданий с лимитом

Формат: Консоль

Уровень: Уровень 2

Ключевые темы: `asyncio.Semaphore`, `bounded concurrency`, сбор статистики.

Цель

Научиться запускать много задач, но удерживать число одновременно активных операций под контролем.

Что реализовать

- Создайте `limited_runner.py`.
- Опишите список из 20 заданий с разной длительностью.
- Реализуйте корутину `run_job(job, semaphore)`, которая входит в `async with semaphore`, печатает старт, ждёт и печатает завершение.
- Сделайте параметр `--limit`, задающий максимальное число активных задач.
- После завершения выведите общее время и максимальное наблюдаемое число одновременно активных задач.

Обязательные требования

- Лимит должен быть положительным целым числом.
- Семафор должен ограничивать именно активную часть работы, а не создание task-ов.
- Счётчик активных задач нужно защищать от логических ошибок: увеличивать и уменьшать в `try/finally`.

Ожидаемое поведение

- При `--limit 1` выполнение похоже на последовательное.
- При `--limit 5` программа быстрее, но одновременно работает не более пяти задач.
- Итоговая статистика подтверждает соблюдение лимита.

Проверка

- Запустить `python limited_runner.py --limit 3`.
- Повторить с лимитами 1, 5 и 20.
- Проверить, что максимум активных задач никогда не превышает лимит.

Критерии оценки

- Правильное применение `Semaphore`.
- Надёжная статистика активных задач.
- Понятная CLI-валидация.

Усложнение

Добавьте сохранение итоговой статистики в JSON-файл через обычный синхронный файл после завершения event loop.

Задание 7. Консольный планировщик напоминаний

Формат: Консоль	Уровень: Уровень 2	Ключевые темы: Отмена задач, timeout, CanceledError, graceful shutdown.
-----------------	--------------------	---

Цель

Разобраться, как отменять запланированные корутины и корректно освобождать управление при завершении программы.

Что реализовать

- Создайте `reminder_scheduler.py`.
- Программа должна принимать команды из stdin: ``add <seconds> <text>``, ``cancel <id>``, ``list``, ``exit``.
- Для каждого напоминания создавайте отдельную task, которая ждёт указанное время и печатает текст.
- Команда ``cancel`` должна отменять task по идентификатору.
- Команда ``exit`` должна отменять все незавершённые напоминания и завершать программу.

Обязательные требования

- Ожидание ввода из stdin перенесите в ``asyncio.to_thread(input, ...)``, чтобы не блокировать event loop.
- Каждая task напоминания должна обрабатывать ``asyncio.CancelledError`` и печатать, что напоминание отменено.
- Для некорректных команд выводите справку, а не traceback.

Ожидаемое поведение

- Можно добавить несколько напоминаний с разными задержками.
- Отменённое напоминание не печатает основной текст.
- При выходе нет предупреждений о pending task.

Проверка

- Запустить программу и добавить три напоминания.
- Отменить одно до срабатывания.
- Выйти до срабатывания последнего и проверить корректное завершение.

Критерии оценки

- Правильная модель отмены.
- Нет блокировки event loop на пользовательском вводе.
- Аккуратная обработка пользовательских ошибок.

Усложнение

Добавьте команду ``cancel_all`` и краткий итог: сколько напоминаний сработало, отменено и осталось активным.

Задание 8. Консольный TCP echo-сервер

Формат: Консоль	Уровень: Уровень 2	Ключевые темы: <code>asyncio.start_server</code> , <code>StreamReader</code> , <code>StreamWriter</code> , несколько клиентов.
-----------------	--------------------	--

Цель

Познакомиться с асинхронными сетевыми streams и обслуживанием нескольких TCP-клиентов в одном процессе.

Что реализовать

- Создайте ``echo_server.py``.
- Поднимите сервер на ``127.0.0.1:8888`` через ``asyncio.start_server``.
- Для каждого клиента читайте строки через ``reader.readline()`` и отправляйте обратно строку с префиксом ``echo:``.
- Если клиент отправил ``quit``, закройте соединение с ним.
- Логируйте подключение, каждое сообщение и отключение клиента.

Обязательные требования

- После записи в ``writer`` обязательно вызывайте ``await writer.drain()``.
- Закрывайте соединение через ``writer.close()`` и ``await writer.wait_closed()``.
- Сервер должен обслуживать несколько клиентов одновременно.

Ожидаемое поведение

- Два клиента могут обмениваться сообщениями независимо.
- Отключение одного клиента не останавливает сервер.
- Команда ``quit`` закрывает только текущее соединение.

Проверка

- Запустить ``python echo_server.py``.
- Подключиться из двух терминалов через ``nc 127.0.0.1 8888``.
- Проверить ответы, отключение одного клиента и работу второго.

Критерии оценки

- Корректная работа `StreamReader/StreamWriter`.
- Нет утечек соединений при нормальном завершении клиента.
- Понятные логи жизненного цикла клиента.

Усложнение

Добавьте ограничение максимальной длины сообщения и закрывайте соединение при превышении лимита.

Задание 9. FastAPI пакетная обработка

Формат: FastAPI	Уровень: Уровень 2	Ключевые темы: POST /batch, gather, semaphore, частичные ошибки.
-----------------	--------------------	--

Цель

Реализовать endpoint, который конкурентно обрабатывает список элементов, но ограничивает нагрузку внутри запроса.

Что реализовать

- Создайте `batch_api.py`.
- Добавьте `POST /batch`, принимающий JSON `{"items": [...], "limit": 3}`.
- Для каждого элемента выполните асинхронную имитацию обработки: задержка зависит от значения элемента.
- Ограничьте одновременную обработку через `asyncio.Semaphore(limit)`.
- Верните список результатов в том же порядке, что и входные элементы.

Обязательные требования

- `limit` должен быть от 1 до 10.
- Пустой список элементов должен отклоняться HTTP 400.
- Если один элемент некорректен, верните ошибку для него в общем результате, не роняя весь batch.

Ожидаемое поведение

- Endpoint обрабатывает несколько элементов быстрее последовательного варианта.
- Порядок результатов соответствует порядку входа.
- Некорректные элементы помечены в ответе отдельно.

Проверка

- Запустить `uvicorn batch_api:app --reload`.
- Отправить batch из 8 элементов с `limit=2` и `limit=5`.
- Сравнить длительность ответа и структуру результата.

Критерии оценки

- Семафор расположен внутри обработки batch.
- Есть предсказуемая схема JSON-ответа.
- Ошибки отдельных элементов не ломают весь запрос.

Усложнение

Добавьте поле ``took_ms`` для каждого элемента и для всего batch.

Задание 10. FastAPI сервис проверки URL

Формат: FastAPI	Уровень: Уровень 2	Ключевые темы: <code>asyncio.to_thread</code> , <code>urllib.request</code> , <code>timeout</code> , защита event loop.
-----------------	--------------------	---

Цель

Понять, как интегрировать блокирующую стандартную библиотеку в асинхронное web-приложение без остановки event loop.

Что реализовать

- Создайте ``url_check_api.py``.
- Добавьте ``POST /check-url``, принимающий ``{"url": "https://..."}``.
- Проверяйте URL через ``urllib.request.urlopen`` с коротким timeout.
- Запускайте блокирующую проверку через ``await asyncio.to_thread(...)``.
- Возвращайте статус доступности, HTTP-код при наличии и длительность проверки.

Обязательные требования

- Разрешите только URL со схемой ``http`` или ``https``.
- Не вызывайте ``urllib.request.urlopen`` напрямую внутри ``async def``.
- Ошибки сети должны превращаться в JSON-ответ, а не в traceback.

Ожидаемое поведение

- Проверка работающего URL возвращает ``available: true``.
- Проверка несуществующего домена возвращает ``available: false``.
- ``/ping``, если он добавлен, отвечает во время долгой проверки URL.

Проверка

- Запустить ``uvicorn url_check_api:app --reload``.
- Проверить ``https://example.com`` и заведомо неверный домен.
- Добавить временный лог, показывающий, что web endpoint не блокируется.

Критерии оценки

- Правильное применение ``asyncio.to_thread``.
- Валидация схемы URL.
- Безопасная обработка сетевых исключений.

Усложнение

Добавьте ``POST /check-urls`` для пакетной проверки нескольких URL с семафором.

Уровень 3. Координация и устойчивость

Задания требуют продуманной обработки ошибок, retry, backpressure, фоновых worker-ов, streaming и простого rate limiting.

Задание 11. Консольный монитор доступности сайтов

Формат: Консоль

Уровень: Уровень 3

Ключевые темы: Retry, timeout, semaphore, итоговый отчёт.

Цель

Собрать устойчивый I/O-bound монитор, который ограничивает нагрузку, повторяет временные ошибки и формирует отчёт.

Что реализовать

- Создайте `site_monitor.py`.
- Программа читает список URL из текстового файла, по одному URL на строку.
- Каждый URL проверяется через `urllib.request` внутри `asyncio.to_thread`.
- Ограничьте число одновременных проверок семафором.
- Добавьте retry: до трёх попыток на URL с небольшой асинхронной паузой между попытками.
- В конце напечатайте таблицу: URL, статус, число попыток, длительность, ошибка.

Обязательные требования

- Одна зависшая проверка не должна ждать бесконечно: задайте timeout для `urlopen` и общий `asyncio.timeout` на задачу.
- Пустые строки и строки с `#` в файле игнорируются.
- Итоговый отчёт должен печататься даже при ошибках отдельных URL.

Ожидаемое поведение

- Рабочие URL отмечаются как доступные.
- Ошибочные URL имеют понятное описание ошибки.
- Общее выполнение использует конкурентность, но соблюдает лимит.

Проверка

- Создать файл `urls.txt` с 5-7 адресами.
- Запустить `python site_monitor.py urls.txt --limit 3`.
- Проверить, что программа не падает из-за одного плохого URL.

Критерии оценки

- Сочетание `Semaphore`, `timeout` и `retry`.
- Отчёт сохраняет полезную диагностическую информацию.
- Чёткое разделение проверки одного URL и orchestration-логики.

Усложнение

Добавьте опцию `--json-report report.json`, которая сохраняет тот же отчёт в JSON.

Задание 12. Pipeline обработки логов

Формат: Консоль

Уровень: Уровень 3

Ключевые темы: Queue, worker pool, backpressure, агрегатор результатов.

Цель

Построить асинхронный конвейер, где чтение, обработка и агрегация разделены очередями.

Что реализовать

- Создайте `log_pipeline.py`.
- Первая корутина читает строки из лог-файла через `asyncio.to_thread` порциями и кладёт их в очередь.
- Несколько worker-ов разбирают строки, определяют уровень `'INFO'`, `'WARNING'`, `'ERROR'` и кладут результат во вторую очередь.
- Агрегатор считает количество строк каждого уровня и топ-5 самых частых сообщений об ошибках.
- После завершения pipeline печатает сводку.

Обязательные требования

- Обе очереди должны иметь `'maxsize'`, чтобы producer не мог бесконечно накапливать данные.
- Завершение worker-ов должно быть явным через `sentinel`.
- Ошибка разбора одной строки не должна останавливать pipeline.

Ожидаемое поведение

- Большой файл обрабатывается без загрузки целиком в память.
- Количество обработанных строк совпадает с числом непустых строк входного файла.
- Итоговая статистика печатается один раз после завершения всех worker-ов.

Проверка

- Подготовить тестовый лог на 50-100 строк.
- Запустить `python log_pipeline.py app.log --workers 4`.
- Сравнить итоговые счётчики с ручной выборкой нескольких строк.

Критерии оценки

- Правильная архитектура `producer -> workers -> aggregator`.
- Нет зависаний на очередях.
- Есть backpressure через ограниченный размер очередей.

Усложнение

Добавьте метрику средней скорости обработки строк в секунду.

Задание 13. FastAPI in-memory job queue

Формат: FastAPI

Уровень: Уровень 3

Ключевые темы: Startup/shutdown, background worker, Queue, статусы jobs.

Цель

Создать web-сервис с собственной очередью задач и фоновой обработкой внутри процесса FastAPI.

Что реализовать

- Создайте `job_queue_api.py`.
- На startup создайте `asyncio.Queue` и запустите одну worker-task.
- Добавьте `POST /jobs`, который создаёт job, кладёт её ID в очередь и возвращает статус `'queued'`.
- Добавьте `GET /jobs/{job_id}`, который возвращает статус: `'queued'`, `'running'`, `'done'` или `'failed'`.
- Worker должен имитировать долгую обработку через `asyncio.sleep` и записывать результат в словарь jobs.
- На shutdown корректно отменяйте worker-task.

Обязательные требования

- Состояние jobs хранится в памяти процесса.
- Worker не должен запускаться заново на каждый запрос.
- На неизвестный job ID возвращайте HTTP 404.
- Отмена worker-а при shutdown должна обрабатываться без traceback.

Ожидаемое поведение

- После создания job сначала виден статус `'queued'` или `'running'`, позже `'done'`.
- Несколько jobs обрабатываются по очереди.
- Сервис корректно завершает фоновые задачи при остановке Uvicorn.

Проверка

- Запустить `uvicorn job_queue_api:app --reload`.
- Создать несколько jobs через `/docs`.
- Периодически запрашивать статус каждой job.
- Остановить сервер и проверить отсутствие предупреждений о pending task.

Критерии оценки

- Корректный lifecycle FastAPI.
- Worker отделён от request handler-ов.
- Статусы jobs обновляются предсказуемо.

Усложнение

Добавьте двух worker-ов и поле `'worker_id'` в результате job.

Задание 14. FastAPI streaming endpoint

Формат: FastAPI

Уровень: Уровень 3

Ключевые темы: StreamingResponse, асинхронный генератор, постепенная отдача данных.

Цель

Научиться отдавать клиенту поток данных, не собирая весь ответ заранее.

Что реализовать

- Создайте `stream_api.py`.
- Добавьте `GET /stream/numbers`, который постепенно отдаёт числа от 1 до `count`.
- Реализуйте асинхронный генератор, который делает `await asyncio.sleep(interval)` между элементами.
- Верните поток через `StreamingResponse` с типом `text/plain`.
- Добавьте `GET /stream/events`, который отдаёт строки в формате Server-Sent Events: `data: ...\n\n`.

Обязательные требования

- `count` должен быть от 1 до 100.
- `interval` должен быть от 0 до 2 секунд.
- Генератор должен иметь блок `finally`, где логируется завершение или разрыв потока.

Ожидаемое поведение

- Клиент получает элементы постепенно.
- При остановке клиента сервер не падает.
- SSE endpoint можно открыть в браузере и увидеть постепенное появление событий.

Проверка

- Запустить `uvicorn stream_api:app --reload`.
- Выполнить `curl -N 'http://127.0.0.1:8000/stream/numbers?count=5&interval=1'`.
- Проверить, что строки появляются с паузами.

Критерии оценки

- Корректный асинхронный генератор.
- Настоящий streaming, а не заранее собранная строка.
- Валидация параметров и аккуратное завершение генератора.

Усложнение

Добавьте endpoint `/stream/progress`, который отдаёт проценты выполнения долгой операции.

Задание 15. FastAPI сервис с rate limit

Формат: FastAPI

Уровень: Уровень 3

Ключевые темы: In-memory rate limiting, monotonic time, HTTP 429.

Цель

Реализовать простое ограничение частоты запросов без внешних хранилищ.

Что реализовать

- Создайте `rate_limit_api.py`.
- Добавьте защищённый endpoint `GET /limited`.
- Идентифицируйте пользователя по заголовку `'X-User'`, а при его отсутствии по IP-адресу клиента.
- Разрешите не более 5 запросов за 10 секунд для одного пользователя.
- При превышении лимита возвращайте HTTP 429 и время до следующей попытки.

Обязательные требования

- Используйте `time.monotonic()`, а не календарное время.
- Старые timestamps нужно очищать, чтобы словарь не рос бесконечно.
- Доступ к структуре лимитера защитите `asyncio.Lock`, потому что инвариант обновляется через несколько действий.

Ожидаемое поведение

- Первые 5 запросов проходят.
- 6-й запрос в пределах окна получает 429.
- После ожидания окно очищается, и запросы снова проходят.

Проверка

- Запустить `uvicorn rate_limit_api:app --reload`.
- Выполнить несколько запросов через `curl -H 'X-User: alice'`.
- Проверить независимые лимиты для `'alice'` и `'bob'`.

Критерии оценки

- Корректная sliding-window или близкая к ней логика.
- HTTP 429 с полезной диагностикой.
- Нет внешних зависимостей и базы данных.

Усложнение

Добавьте `GET /rate-limit/status`, который показывает остаток запросов для текущего пользователя.

Уровень 4. Комплексные мини-проекты

Финальные задачи объединяют несколько паттернов в маленькие, но целостные приложения с жизненным циклом, отменой и наблюдаемым состоянием.

Задание 16. Консольный многоклиентский чат

Формат: Консоль

Уровень: Уровень 4

Ключевые темы: TCP streams, broadcast, множество клиентов, graceful shutdown.

Цель

Собрать полноценное сетевое приложение, где каждый клиент получает сообщения остальных клиентов.

Что реализовать

- Создайте `chat_server.py`.
- Сервер принимает TCP-клиентов на `127.0.0.1:7777`.
- При подключении клиент отправляет имя первой строкой.
- Каждое последующее сообщение рассылается всем остальным подключённым клиентам.
- При отключении клиента сервер уведомляет остальных.
- По `Ctrl+C` сервер закрывает все соединения и завершает задачи клиентов.

Обязательные требования

- Храните клиентов в структуре с `writer`-ами и именами.
- Операцию рассылки защитите от изменения списка клиентов во время отправки.
- Ошибки записи одному клиенту не должны ломать рассылку всем остальным.
- Для каждого `writer` вызывайте `drain`, `close` и `wait_closed`.

Ожидаемое поведение

- Три клиента видят сообщения друг друга.
- Отключение одного клиента не влияет на остальных.
- При остановке сервера клиенты получают закрытие соединения, а процесс завершается без висящих задач.

Проверка

- Запустить сервер.
- Подключить 3 клиента через `nc 127.0.0.1 7777`.
- Проверить вход, рассылку сообщений, отключение клиента и остановку сервера.

Критерии оценки

- Рабочий broadcast без падений на отключениях.
- Корректная очистка ресурсов.
- Читаемая архитектура: подключение, чтение, рассылка и shutdown разделены.

Усложнение

Добавьте личные сообщения формата ``/w username text``.

Задание 17. Консольный оркестратор задач

Формат: Консоль	Уровень: Уровень 4	Ключевые темы: <code>asyncio.TaskGroup</code> , зависимые задачи, отмена дерева задач, итоговый лог.
-----------------	--------------------	--

Цель

Освоить структурированную конкурентность и поведение группы задач при ошибке одной из них.

Что реализовать

- Создайте `task_orchestrator.py` для Python 3.11+.
- Опишите workflow из нескольких стадий: загрузка данных, валидация, обработка частей, сбор результата.
- Стадия обработки должна запускать несколько подзадач внутри `asyncio.TaskGroup`.
- Одна из подзадач в демонстрационном режиме должна уметь падать с исключением.
- Соберите итоговый лог: какие стадии начались, завершились, были отменены или упали.

Обязательные требования

- Используйте `asyncio.TaskGroup`, а не только `gather`.
- Каждая подзадача должна иметь `try/finally` для записи события завершения или отмены.
- Исключение одной подзадачи должно отменять соседние задачи внутри группы.
- Для Python ниже 3.11 в README к заданию опишите, что требуется обновить Python.

Ожидаемое поведение

- В нормальном режиме workflow завершается итоговым результатом.
- В режиме ошибки соседние подзадачи отменяются, а ошибка попадает в лог.
- Лог помогает восстановить порядок событий.

Проверка

- Запустить `python task_orchestrator.py --mode ok`.
- Запустить `python task_orchestrator.py --mode fail`.
- Проверить, что при fail видна отмена задач внутри группы.

Критерии оценки

- Корректное применение структурированной конкурентности.
- Понятная модель распространения ошибки.
- Итоговый лог пригоден для диагностики.

Усложнение

Добавьте экспорт лога в JSON Lines файл.

Задание 18. FastAPI realtime-лента событий

Формат: FastAPI

Уровень: Уровень 4

Ключевые темы: Публикация событий, подписка через streaming, очереди подписчиков.

Цель

Построить простую realtime-систему: одни запросы публикуют события, другие получают их потоком.

Что реализовать

- Создайте `realtime_feed_api.py`.
- Добавьте `POST /events`, который принимает событие с типом и текстом.
- Добавьте `GET /events/stream`, который отдаёт события в формате SSE.
- Для каждого подключённого подписчика создавайте отдельную `asyncio.Queue`.
- При публикации события кладите его во все активные очереди подписчиков.
- При отключении подписчика удаляйте его очередь.

Обязательные требования

- Очередь подписчика должна иметь `maxsize`; если подписчик не успевает, старые события можно пропускать с логом.
- Разрыв клиентского соединения не должен оставлять подписчика в памяти.
- События должны включать `id`, `type`, `text` и `timestamp`.

Ожидаемое поведение

- Открытый stream получает новые события без перезагрузки страницы.
- Несколько подписчиков получают одно и то же событие.
- После отключения подписчика публикации продолжают работать.

Проверка

- Запустить `uvicorn realtime_feed_api:app --reload`.
- Открыть stream через `curl -N` в двух терминалах.
- Отправить события через Swagger UI или curl и проверить доставку.

Критерии оценки

- Корректная регистрация и удаление подписчиков.
- SSE-формат соблюден.
- Backpressure для медленных подписчиков учтён явно.

Усложнение

Добавьте `GET /events/recent`, который возвращает последние 20 опубликованных событий.

Задание 19. FastAPI асинхронный кеш

Формат: FastAPI

Уровень: Уровень 4

Ключевые темы: TTL, `asyncio.Lock`, cache stampede protection, reset endpoint.

Цель

Реализовать кеш для дорогой операции так, чтобы при одновременных запросах она выполнялась один раз.

Что реализовать

- Создайте `async_cache_api.py`.
- Добавьте `GET /data/{key}`, который возвращает значение из кеша или вычисляет его через медленную корутину.
- Кешируйте значение на 30 секунд.
- Защитите вычисление одного ключа через `lock`, чтобы десять одновременных запросов к новому ключу не запустили десять вычислений.
- Добавьте `DELETE /cache/{key}` и `DELETE /cache`, чтобы сбрасывать один ключ или весь кеш.

Обязательные требования

- Используйте `time.monotonic()` для TTL.
- Lock должен быть на уровне ключа или аккуратно минимизировать глобальную блокировку.
- Ответ должен показывать `cache: hit|miss` и время вычисления.
- Не используйте Redis или внешнюю базу.

Ожидаемое поведение

- Первый запрос к ключу медленный и помечен как `miss`.
- Повторный запрос до истечения TTL быстрый и помечен как `hit`.
- Параллельные первые запросы к одному ключу приводят к одному вычислению.
- После удаления ключ снова вычисляется.

Проверка

- Запустить `uvicorn async_cache_api:app --reload`.
- Отправить несколько одновременных запросов к одному ключу.
- Проверить логи вычислений и ответы `hit/miss`.

Критерии оценки

- Корректная TTL-логика.
- Защита от `cache stampede`.
- Сброс кеша работает без повреждения состояния.

Усложнение

Добавьте статистику `GET /cache/stats`: число `hits`, `misses`, `evictions` и активных ключей.

Задание 20. Итоговый FastAPI Async Job Lab

Формат: FastAPI	Уровень: Уровень 4	Ключевые темы: Job queue, worker pool, отмена jobs, stream прогресса, health endpoint.
-----------------	--------------------	--

Цель

Собрать итоговое приложение, объединяющее очереди, worker pool, статусы, отмену и streaming прогресса.

Что реализовать

- Создайте `async_job_lab.py`.
- Добавьте `POST /jobs`, который создаёт job с параметрами `steps` и `delay`.
- Запустите worker pool на startup: минимум 3 worker-task, читающие IDs jobs из общей очереди.
- Каждый worker выполняет job по шагам, обновляя progress от 0 до 100.
- Добавьте `GET /jobs/{job_id}` для статуса и результата.
- Добавьте `POST /jobs/{job_id}/cancel`, который отменяет job, если она ещё не завершена.
- Добавьте `GET /jobs/{job_id}/events`, который stream-ит прогресс job через SSE.
- Добавьте `GET /health`, показывающий размер очереди, число worker-ов и число jobs по статусам.

Обязательные требования

- Статусы jobs: `queued`, `running`, `done`, `failed`, `cancelled`.
- Отмена queued job должна не позволить worker-у начать её работу.
- Отмена running job должна быть замечена на ближайшем шаге.
- Подписчики progress stream должны удаляться при отключении.
- На shutdown worker-ы отменяются корректно, без pending task warning.

Ожидаемое поведение

- Несколько jobs выполняются параллельно разными worker-ами.
- Progress stream показывает движение по шагам.
- Отмена меняет статус и останавливает дальнейшую обработку.
- `/health` отражает реальное состояние приложения.

Проверка

- Запустить `uvicorn async_job_lab:app --reload`.
- Создать 5 jobs с разной длительностью.
- Открыть stream прогресса одной job через `curl -N`.
- Отменить одну queued и одну running job.
- Проверить статусы, health и отсутствие висящих подписчиков после отключения stream.

Критерии оценки

- Целостная архитектура приложения.
- Корректная отмена и жизненный цикл worker pool.
- Streaming progress работает независимо от polling статуса.
- Код остаётся читаемым и разбитым на функции вместо большого endpoint-a.

Усложнение

Добавьте простую HTML-страницу `/ui`, которая показывает список jobs и прогресс без сторонних frontend-зависимостей.

Критерии итоговой оценки

Критерий	Что оценивается
Корректность	Программа выполняет заявленный сценарий, возвращает ожидаемые статусы, не зависает и не теряет данные при нормальной нагрузке.
Асинхронная модель	Используются `async def`, `await`, задачи и примитивы `asyncio` по назначению; блокирующие операции не выполняются напрямую в event loop.
Управление ошибками	Ошибки ввода, сетевые сбои, тайм-ауты и отмена задач обрабатываются явно и не превращаются в необъяснимый traceback.
Жизненный цикл	Фоновые задачи, TCP-соединения, очереди, подписчики и worker-ы корректно стартуют и завершаются.
Наблюдаемость	Вывод, логи или JSON-ответы помогают понять состояние приложения, длительность операций и причину отказа.
Читаемость	Код разбит на функции, имена отражают смысл, отсутствуют большие endpoint-ы или корутины, делающие всё сразу.

Приложение. Команды запуска и проверки

Базовые команды для консольных заданий:

```
python -m compileall .
python async_timer.py
python limited_runner.py --limit 3
python site_monitor.py urls.txt --limit 3
```

Базовые команды для FastAPI-заданий:

```
python -m pip install fastapi uvicorn
uvicorn delay_api:app --reload
uvicorn job_queue_api:app --reload
curl -N http://127.0.0.1:8000/stream/numbers?count=5&interval=1
```

Минимальный чек-лист перед сдачей:

- Программа запускается из чистого терминала по инструкции из README.
- Основной сценарий и минимум один ошибочный сценарий проверены вручную.
- Для FastAPI endpoint-ов проверены HTTP-коды и JSON-ответы.
- В асинхронных корутинах нет `time.sleep` и прямых блокирующих сетевых вызовов.
- После остановки приложения нет предупреждений о незавершённых задачах.